Dynamic Programming & Linked Lists Cheatsheet

From Fundamentals to Optimization

• Core Concepts

References in Java

Objects: Pass by reference (shared)
Primitives: Pass by value (copied)

```
ListNode a = new ListNode(1);
ListNode b = a; // Same object!
b.val = 5;
// a.val is now 5 too
```

DP Pattern Recognition

When to use DP:
✓ Optimal substructure
✓ Overlapping subproblems
✓ Can break into smaller problems
✓ Choices at each step

Top-Down vs Bottom-Up

Top-Down (Memoization):
- Start from problem → base cases
- Uses recursion + cache
- More intuitive, lazy evaluation

Bottom-Up (Tabulation):
- Start from base cases → answer
- Uses iteration + DP array
- More efficient, no stack overflow

○ Linked List Problems

1. Reverse Linked List

Problem: Reverse a singly linked list.
`Input: 12345` `Output: 54321`

Approach 1: Iterative
Time: O(n) **Space:** O(1)
Key Idea: Use 3 pointers, flip arrows as you go.
Variables Needed:

- `prev` = null (new tail)
- `current` = head (node we're working on)
- `nextTemp` = save next before breaking link

```
ListNode reverseList(ListNode head) {
    ListNode prev = null;
    ListNode current = head;

    while (current != null) {
        // Save next
        ListNode nextTemp = current.next;
        // Reverse pointer
        current.next = prev;
        // Move forward
        prev = current;
        current = nextTemp;
    }
    return prev; // New head
}
```

Common Mistakes:

- × Forgetting to save `nextTemp` → lose rest of list
- × Returning `head` instead of `prev`

Approach 2: Recursive
Time: O(n) **Space:** O(n) *(call stack)*
Key Idea: Reverse rest first, then fix current node.

```
ListNode reverseList(ListNode head) {
    // Base case
    if (head == null || head.next == null)
        return head;

    // Reverse rest
    ListNode newHead = reverseList(head.next);

    // Fix pointers
    head.next.next = head; // Flip arrow
    head.next = null;      // Break forward

    return newHead; // Pass up new head
}
```

Execution:
↓ Go to last node (base case)
↑ Fix pointers coming back up
Work happens AFTER recursion returns

---

2. Merge Two Sorted Lists

Problem: Merge two sorted lists into one sorted list.
`L1: 135` `L2: 246`
`Output: 123456`

Approach 1: Iterative (Dummy Node)
Time: O(n+m) **Space:** O(1)
Key Idea: Compare heads, attach smaller, move forward.
Dummy Node Trick: Create fake starting node to avoid special cases.

```
ListNode mergeTwoLists(ListNode l1, ListNode l2) {
    ListNode dummy = new ListNode(0);
    ListNode node = dummy;

    while (l1 != null && l2 != null) {
        if (l1.val < l2.val) {
            node.next = l1;
            l1 = l1.next;
        } else {
            node.next = l2;
            l2 = l2.next;
        }
        node = node.next;
    }

    // Attach remaining
    node.next = (l1 != null) ? l1 : l2;

    return dummy.next; // Skip dummy
}
```

Why dummy.next works:
`dummy` and `node` initially point to same object. As `node` moves, they stay connected via `.next` chain. `dummy` never moves, so `dummy.next` is the start of merged list.

Approach 2: Recursive
Time: O(n+m) **Space:** O(n+m) *(call stack)*
Key Idea: Pick smaller head, recursively merge rest.

```
ListNode mergeTwoLists(ListNode l1, ListNode l2) {
    // Base cases
    if (l1 == null) return l2;
    if (l2 == null) return l1;

    // Recursive case
    if (l1.val < l2.val) {
        l1.next = mergeTwoLists(l1.next, l2);
        return l1;
    } else {
        l2.next = mergeTwoLists(l1, l2.next);
        return l2;
    }
}
```

⋆ Dynamic Programming Problems

3. Climbing Stairs

Problem: Ways to climb `n` stairs taking 1 or 2 steps.
`n=3:` Ways = 3 (1+1+1, 1+2, 2+1)
Pattern: This is Fibonacci!
`stairs(n) = stairs(n-1) + stairs(n-2)`
Why? To reach step `n`, you came from:
- Step `n-1` (take 1 step) OR
- Step `n-2` (take 2 steps)

Approach 1: Pure Recursion
Time: O(2^n) **Space:** O(n) × *TOO SLOW!*

```
int climbStairs(int n) {
    if (n == 0 || n == 1) return 1;
    return climbStairs(n-1) + climbStairs(n-2);
}
```

Problem: Repeated calculations!
For `n=40`: ∼1 trillion calls!

Approach 2: Memoization (Top-Down)

Time: O(n) **Space:** O(n)
Key Idea: Cache results to avoid recalculation.

```
int climbStairs(int n) {
    int[] memo = new int[n + 1];
    return helper(n, memo);
}

int helper(int n, int[] memo) {
    // Base cases
    if (n == 0 || n == 1) return 1;

    // Check cache
    if (memo[n] != 0) return memo[n];

    // Calculate & store
    memo[n] = helper(n-1, memo)
            + helper(n-2, memo);

    return memo[n];
}
```

Critical: Create memo ONCE, pass to all calls!
× Don't create memo inside recursive function!

Approach 3: Iterative DP (Bottom-Up)

Time: O(n) **Space:** O(n)
Key Idea: Build from base cases up to answer.

```
int climbStairs(int n) {
    if (n <= 1) return 1;

    int[] dp = new int[n + 1];
    dp[0] = 1; // Base case
    dp[1] = 1; // Base case

    for (int i = 2; i <= n; i++) {
        dp[i] = dp[i-1] + dp[i-2];
    }

    return dp[n];
}
```

DP Table Example (n=5):

i	0	1	2	3	4	5
dp[i]	1	1	2	3	5	8

Approach 4: Space-Optimized

Time: O(n) **Space:** O(1) ✓ *OPTIMAL!*
Key Idea: Only need last 2 values!

```
int climbStairs(int n) {
    if (n <= 1) return 1;

    int prev2 = 1; // dp[i-2]
    int prev1 = 1; // dp[i-1]

    for (int i = 2; i <= n; i++) {
        int current = prev1 + prev2;
        prev2 = prev1; // Slide window
        prev1 = current;
    }

    return prev1;
}
```

Understanding prev1/prev2:
When calculating step 5:
`prev1` = ways to reach step 4
`prev2` = ways to reach step 3

Extension: 3 Step Sizes [1,2,3]

Formula: `stairs(n) = stairs(n-1) + stairs(n-2) + stairs(n-3)`
Base Cases: Need 3 now!

```
if (n == 0) return 1;
if (n == 1) return 1;
if (n == 2) return 2;

// For n >= 3:
dp[i] = dp[i-1] + dp[i-2] + dp[i-3];
```

Pattern: k step sizes → k base cases → k terms in formula

---

4. House Robber

Problem: Rob houses for max money, can't rob adjacent.
`nums = [2,7,9,3,1]` → Answer: 12 (rob 0,2,4)
Key Insight: At each house, make optimal choice!
NOT a fixed even/odd pattern!
Recurrence:
`dp[i] = max(rob house i, skip house i)`
`dp[i] = max(nums[i] + dp[i-2], dp[i-1])`
Why?

- Rob house `i`: Get `nums[i]` + best from `i-2`
- Skip house `i`: Keep best from `i-1`

Approach 1: Iterative DP

Time: O(n) **Space:** O(n)

```
int rob(int[] nums) {
    if (nums.length == 1) return nums[0];

    int[] dp = new int[nums.length];
    dp[0] = nums[0];
    dp[1] = Math.max(nums[0], nums[1]);

    for (int i = 2; i < nums.length; i++) {
        dp[i] = Math.max(
            nums[i] + dp[i-2], // Rob this
            dp[i-1]            // Skip this
        );
    }

    return dp[nums.length - 1];
}
```

DP Table Example:

i	0	1	2	3	4
nums[i]	2	7	9	3	1
dp[i]	2	7	11	11	12

Approach 2: Memoization (Top-Down)

Time: O(n) **Space:** O(n)

Direction: Start from index 0, go forward to end.

```
int rob(int[] nums) {
    int[] memo = new int[nums.length];
    Arrays.fill(memo, -1);
    return robFrom(0, nums, memo);
}

int robFrom(int i, int[] nums, int[] memo) {
    // Base case: past last house
    if (i >= nums.length) return 0;

    // Check cache
    if (memo[i] != -1) return memo[i];

    // Calculate: rob vs skip
    int robCurrent = nums[i]
                   + robFrom(i+2, nums, memo);
    int skipCurrent = robFrom(i+1, nums, memo);

    memo[i] = Math.max(robCurrent, skipCurrent);
    return memo[i];
}
```

Parameter meaning:
`robFrom(i, nums)` = "Max money from house `i` to end"
`i` = index in `nums` array
Must pass `nums` so function can access `nums[i]`

Approach 3: Space-Optimized

Time: O(n) **Space:** O(1)

```
int rob(int[] nums) {
    if (nums.length == 1) return nums[0];

    int prev2 = nums[0];
    int prev1 = Math.max(nums[0], nums[1]);

    for (int i = 2; i < nums.length; i++) {
        int current = Math.max(
            nums[i] + prev2,
            prev1
        );
        prev2 = prev1;
        prev1 = current;
    }

    return prev1;
}
```

▷ Code Templates

Memoization Template

```
ReturnType solve(params) {
    Cache memo = new Cache();
    return helper(params, memo);
}

ReturnType helper(params, Cache memo) {
    // Base case
    if (baseCondition) return baseValue;

    // Check cache
    if (memo.has(key)) return memo.get(key);

    // Calculate
    result = recursive_calculation;

    // Store & return
    memo.put(key, result);
    return result;
}
```

Iterative DP Template

```
ReturnType solve(params) {
    // Initialize DP array
    DP[] dp = new DP[size];

    // Base cases
    dp[0] = baseCase0;
    dp[1] = baseCase1;

    // Fill table
    for (int i = start; i <= end; i++) {
        dp[i] = operation(dp[i-k], ...);
    }

    return dp[end];
}
```

Space-Optimized Template

```
ReturnType solve(params) {
    // Base cases
    if (n <= k) return baseCase;

    // Keep only last k values
    Type prevK = baseK;
    ...
    Type prev1 = base1;

    for (int i = k; i <= n; i++) {
        current = operation(prev1, prev2,
    ...);

        // Slide window
        prevK = prevK-1;
        ...
        prev1 = current;
    }

    return prev1;
}
```

Complexity Analysis

Time Complexity Logic

Pure Recursion: O(2^n)
- Each call makes 2 calls → exponential tree
- Repeated calculations

Memoization: O(n)
- Each subproblem calculated once
- Total subproblems: n
- Work per subproblem: O(1)

Iterative DP: O(n)
- Single loop from start to end
- Each iteration: O(1) work

Space-Optimized: O(n) time, O(1) space
- Same time as iterative
- Only store constant variables

Space Complexity Logic

Recursion: O(n)
- Call stack depth = n
- Each stack frame takes space

Memoization: O(n)
- Memo array: O(n)
- Call stack: O(n)
- Total: O(n) + O(n) = O(n)

Iterative DP: O(n)
- DP array: O(n)
- No call stack

Space-Optimized: O(1)
- Only k variables (constant)
- No array, no stack

Common Mistakes

Memoization Pitfalls

× **Creating memo inside recursive function**
Each call gets NEW array → no sharing → no benefit!

✓ **Solution:** Create once, pass to all calls

```
// WRONG
int solve(int n) {
    int[] memo = new int[100]; // NEW!
    if (memo[n] != 0) return memo[n];
    memo[n] = solve(n-1) + solve(n-2);
    return memo[n];
}

// CORRECT
int solve(int n) {
    int[] memo = new int[n+1];
    return helper(n, memo);
}
int helper(int n, int[] memo) {
    if (memo[n] != 0) return memo[n];
    memo[n] = helper(n-1, memo)
            + helper(n-2, memo);
    return memo[n];
}
```

Linked List Pitfalls

× **Losing reference to next node**

```
// WRONG - loses rest of list
current.next = prev;
current = current.next; // This is prev!
```

✓ **Solution:** Save next before modifying

```
// CORRECT
ListNode nextTemp = current.next;
current.next = prev;
current = nextTemp;
```

× **Returning wrong node**

```
return head; // WRONG in reverse!
```

✓ **Solution:** Return new head (prev)

```
return prev; // CORRECT
```

♢ Key Insights

Base Cases

Base case returns are OUTPUTS, not inputs!
`if (n == 0) return 1` means:
"If INPUT is 0 stairs, OUTPUT is 1 way"
NOT "take 0 steps equals 1 step"

`n == 0` returns 1 because:
"There's 1 way to climb 0 stairs: by already being there"

References

Returning head returns entire list!
`return prev;` in reverse doesn't just return one node. It returns a REFERENCE to a node, which is connected to others via `.next` chain.

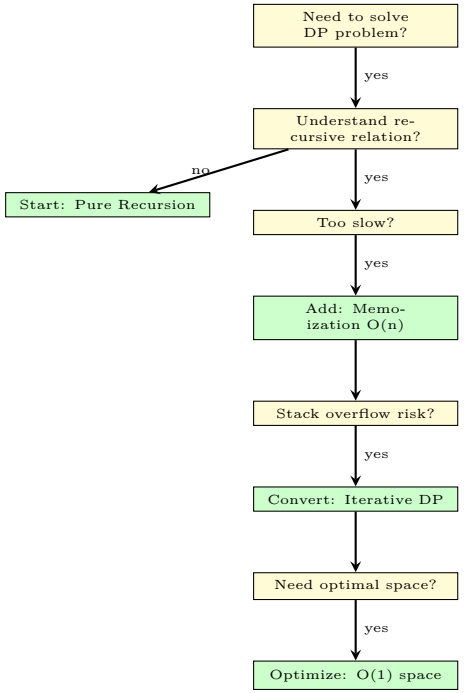
Think: Returning engine of train = returning whole train (cars are connected)

Direction

Recursion can go either direction!
Climbing stairs: Can start at `n` (go down) OR start at `0` (go up)
House robber: Can start at `0` (go forward) OR at `n-1` (go backward)

Both work! Direction is just a framing choice.

♡ Problem-Solving Decision Tree



♣ Quick Reference

Approach	Time	Space	Use
Pure Recursion	O(2^n)	O(n)	Learning
Memoization	O(n)	O(n)	Natural
Iterative DP	O(n)	O(n)	Default
Space-Opt	O(n)	O(1)	Optimal

When to Use Each:

✓ Memoization	Naturally recursive
✓ Iterative DP	Most problems
✓ Space-Opt	Space critical
× Pure Recursion	Never in production

♠ Learning Progression

1. **Understand recursion**
   Base case + recursive case + progress
2. **Identify overlapping subproblems**
   Draw recursion tree, spot duplicates
3. **Add memoization**
   Cache results, check before computing
4. **Convert to iterative**
   Build from base cases, use loop
5. **Optimize space**
   Keep only what you need

Practice Order:

1. Climbing Stairs (basic Fibonacci)
2. House Robber (max choice)
3. Min Cost Climbing Stairs (min choice)
4. Unique Paths (2D grid)
5. Coin Change (unbounded)
6. 0/1 Knapsack (bounded choice)

Remember: Start simple (recursion), optimize gradually (memo → iterative → space)
Practice makes perfect! Trace examples by hand first.